EXACT DISTANCES OF FIVE WEIGHT-SIX TWO-BLOCK CODES

TIMO KELLER

Abstract. We determine the minimum distance of five quantum CSS codes with checks of weight six and qubit degree six: a two-block group algebra code $[[600, 24, 26]]$ over a non-abelian group of order 300, a cyclic two-block code $[[504, 12, 28]]$, and the cyclic two-block codes $[[510, 16, 26]]$, $[[558, 14, 28]]$ and $[[434, 10, 28]]$ of the qLDPC challenge board, whose distances were known there only as upper bounds. The upper bounds are explicit logical operators. The lower bounds are computer-assisted: an exhaustive anchored search with exclusions, whose completeness we prove, run with two independently written programs. Programs, inputs and logs are in the directory `weight6-exact-distances` of `https://github.com/TimoKellerMath/quantum-error-correction`.

1. The codes

Let G be a finite group and $a, b \in \mathbb{F}_2[G]$. The two-block group algebra code $C(G; a, b)$ [WLP23, LP24] has $n = 2|G|$ qubits, two blocks indexed by G, and the parity-check matrices

$$H_X = (L_a \mid R_b), \qquad H_Z = (R_b^{\mathsf{T}} \mid L_a^{\mathsf{T}}),$$

where L_a and R_b are the matrices of left multiplication by a and right multiplication by b in the basis G. Explicitly, the X-check at $c \in G$ has support $\{s^{-1}c : s \in \operatorname{supp} a\}$ in the first block and $\{ct^{-1} : t \in \operatorname{supp} b\}$ in the second, and the Z-check at c has support $\{ct : t \in \operatorname{supp} b\}$ in the first block and $\{sc : s \in \operatorname{supp} a\}$ in the second. Since left and right multiplications commute, $H_X H_Z^{\mathsf{T}} = L_a R_b + R_b L_a = 0$. Every check has weight $|\operatorname{supp} a| + |\operatorname{supp} b|$, since left and right translations are injective, and every qubit lies in $|\operatorname{supp} a| + |\operatorname{supp} b|$ checks: a qubit of the first block in $|\operatorname{supp} a|$ X-checks and $|\operatorname{supp} b|$ Z-checks, a qubit of the second block in $|\operatorname{supp} b|$ X-checks and $|\operatorname{supp} a|$ Z-checks. The number of logical qubits is $k = n - \operatorname{rk} H_X - \operatorname{rk} H_Z$.

For the cyclic codes we use the convention of the qLDPC challenge board [Board]: for $a, b \in \mathbb{F}_2[x]/(x^m - 1)$,

$$H_X = (\operatorname{circ}(a) \mid \operatorname{circ}(b)), \qquad H_Z = (\operatorname{circ}(b)^{\mathsf{T}} \mid \operatorname{circ}(a)^{\mathsf{T}}), \qquad \operatorname{circ}(v)_{i,i+j} = v_j,$$

so the X-check i is $\{i + s : s \in \operatorname{supp} a\} \cup \{m + i + t : t \in \operatorname{supp} b\}$ and the Z-check i is $\{i - t : t \in \operatorname{supp} b\} \cup \{m + i - s : s \in \operatorname{supp} a\}$, indices modulo m within each block. This is $C(\mathbb{Z}/m; a^*, b^*)$ with $a^*(x) = a(x^{-1})$.

name	group	a, b	$[[n, k, d]]$	kd^2/n
C600	$G_{300} \cong D_5 \times C_{30}$	$1 + x^{24}y^8 + x^{13}y^9$, $1 + x^{22}y^4 + x^{24}y^7$	$[[600, 24, 26]]$	27.04
C504	$\mathbb{Z}/252$	$1 + x + x^{58}$, $1 + x^{48} + x^{166}$	$[[504, 12, 28]]$	18.67
B510	$\mathbb{Z}/255$	$1 + x^{26} + x^{76}$, $1 + x^{30} + x^{66}$	$[[510, 16, 26]]$	21.21
B558	$\mathbb{Z}/279$	$1 + x^{37} + x^{140}$, $1 + x^{61} + x^{197}$	$[[558, 14, 28]]$	19.67
B434	$\mathbb{Z}/217$	$1 + x^{29} + x^{117}$, $1 + x^{19} + x^{111}$	$[[434, 10, 28]]$	18.06

Here $G_{300} = \langle x, y \mid x^{30}, y^{10}, yxy^{-1} = x^{19} \rangle$, and in C600 the element $x^i y^j$ is the qubit $10i + j$ of the first block and $300 + 10i + j$ of the second. C600 is a code of type $3 + 3$ over a group of

Date: 10 November 2026.

order 300 which we found in a search over 3-fold voltage lifts, that is covers of codes in the sense of [GZ25, HL26], of a $[[200, 16, 12]]$ code over $D_5 \times C_{10}$. That code has the parameters of the weight-six code over $C_{10} \times D_{10}$ (GAP notation for $D_5 \times C_{10}$) in the census [Lee26, Table 9], credited there to the code archive of [LP24]; we have not checked that the two are equivalent. For a cyclic two-block code $k = 2 \deg \gcd(a, b, x^m - 1)$ [LP24, Eq. (44)]. The code C504 was found in a search for cyclic codes in which this gcd is the minimal polynomial of a root of unity of odd order N; for C504 it is the minimal polynomial of an element of order 63, of degree $\mathrm{ord}_{63}(2) = 6$ (computed with Sage; for B510, B558 and B434 the gcd has irreducible factors whose roots have orders 85; 3 and 31; 31). B510, B558 and B434 are the board entries `510-16-26`, `558-14-28` and `434-10-28`, submitted by V. Russo with witness-backed upper bounds; `src/codes.py --board DIR`, with `DIR` the directory `codes` of the board repository at the commit cited in [Board], checks that our matrices coincide with the board's files (output in `runs/codes_check.log`). The values of n, k, the check weights and the column weights are computed by `src/codes.py`.

2. Minimum-weight logical operators

Let $H_X \in \mathbb{F}_2^{m_X \times n}$ and $H_Z \in \mathbb{F}_2^{m_Z \times n}$ with $H_X H_Z^{\mathsf{T}} = 0$. A Z-logical operator is a vector $z \in \ker H_X$; it is trivial if $z \in \mathrm{rowsp}\, H_Z$, and d_Z is the least weight of a nontrivial one; d_X is defined symmetrically and $d = \min(d_X, d_Z)$. We identify vectors with their supports. For a set T of qubits put $\sigma(T) = H_X 1_T$; an X-check c is unsatisfied by T if $\sigma(T)_c = 1$. A vector $z \in \ker H_X$ is trivial if and only if z is orthogonal to a basis of $\ker H_Z$ modulo $\mathrm{rowsp}\, H_X$, since $\mathrm{rowsp}\, H_Z = (\ker H_Z)^{\perp}$; this is the nontriviality test of both programs.

Lemma 2.1. *Let z be a nontrivial Z-logical operator with $|z| = d_Z$.*

(a) If $\emptyset \neq T \subsetneq z$, then $\sigma(T) \neq 0$.
(b) For every $s \in \mathrm{rowsp}\, H_Z$, $|z \cap s| \leq |s|/2$.
(c) If $T \subseteq z$ and $E = z \setminus T$, then $|E \cap c|$ is odd for every X-check c unsatisfied by T.

Proof. (a) If $\sigma(T) = 0$, then T and $z \setminus T$ lie in $\ker H_X$ and add up to z, so one of them is nontrivial and lighter than z. (b) $z + s$ is nontrivial, so $d_Z \leq |z + s| = |z| + |s| - 2|z \cap s|$. (c) $H_X 1_E = H_X z + \sigma(T) = \sigma(T)$. □

Part (a) is [RD26, Lemma 2], [OK26, Lemma 2] and [CGP26, Suppl. Inf., Lemma 5.1], and for classical codes [DKP17, §V, Lemma 3]; part (b) is the dominance bound of [CGP26, Suppl. Inf., eq. (S42)]; the weaker form of (c), that E meets every check unsatisfied by T, is the remark after [RD26, Lemma 2], and (c) is used in the proofs of [OK26, Thm. 2] and [CGP26, Suppl. Inf., §5.5.1].

Lemma 2.2. *If every column of H_X has odd weight, then every element of $\ker H_X$ has even weight. In particular d_Z is even, and if there is no nontrivial Z-logical operator of weight at most $2w$, then $d_Z \geq 2w + 2$.*

Proof. The sum of the rows of H_X is the all-ones vector, and $z \in \ker H_X$ is orthogonal to it. □

The same reduction is used in [OK26, §V-A], in [LOMK26, App. B.1] and, for the code $[[360, 12, \leq 24]]$ of [B+24], in the fieldnote [Cro26]. In all five codes every column of H_X and of H_Z has weight 3.

Lemma 2.3. *Let π be a permutation of the qubits that maps the set of X-check supports onto the set of Z-check supports and the set of Z-check supports onto the set of X-check supports. Then $d_X = d_Z$. For the cyclic codes of Section 1, $\pi(i) = m + (-i \bmod m)$ and $\pi(m + i) = -i \bmod m$ has this property.*

Proof. π maps $\ker H_X$ onto $\ker H_Z$ and $\operatorname{rowsp} H_Z$ onto $\operatorname{rowsp} H_X$, preserving weights. For the cyclic codes, π maps the X-check i, that is $\{i+s\} \cup \{m+i+t\}$, to $\{m-i-s\} \cup \{-i-t\}$, which is the Z-check $-i$; likewise it maps the Z-check i to the X-check $-i$. □

For bivariate bicycle codes the analogous permutation is the ZX-duality of [B+24].

Lemma 2.4. *Let Γ be a group of qubit permutations that preserve* $\operatorname{rowsp} H_X$ *and* $\operatorname{rowsp} H_Z$*. Let $O_1, \dots, O_r$ be its orbits on the qubits, in any order, $q_i \in O_i$, and $X_i \subseteq O_1 \cup \cdots \cup O_{i-1}$. If $d_Z \le W$, then for some i there is a nontrivial Z-logical operator z with $|z| = d_Z$, $q_i \in z$ and $z \cap X_i = \emptyset$.*

Proof. Let z_0 be a nontrivial Z-logical operator of weight d_Z, i minimal with $z_0 \cap O_i \neq \emptyset$, $p \in z_0 \cap O_i$ and $\gamma \in \Gamma$ with $\gamma p = q_i$. Then $z = \gamma z_0$ is a nontrivial Z-logical operator of weight d_Z containing q_i, and $z \cap O_j = \gamma(z_0 \cap O_j) = \emptyset$ for $j < i$. □

Lemma 2.5. *Let Γ be as in Lemma 2.4, acting simply transitively on the X-checks, all of weight w, and assume that every qubit lies in some X-check. Fix an X-check c_0 and a bijection $\rho_0 \colon \{1, \dots, w\} \to c_0$, put $\rho_{\gamma c_0} = \gamma \circ \rho_0$, and call $\rho_c^{-1}(z \cap c)$ the pattern of z at c. Let $\pi_1, \dots, \pi_M$ be the nonempty subsets of $\{1, \dots, w\}$ of even size, in any order. If $d_Z \le W$, then for some j there is a nontrivial Z-logical operator z with $|z| = d_Z$, pattern π_j at c_0, and no pattern among $\pi_1, \dots, \pi_{j-1}$ at any check.*

Proof. A Z-logical operator meets every X-check in an even set, and a nonzero one meets some X-check, since every qubit lies in one; so every nontrivial Z-logical operator has a pattern among $\pi_1, \dots, \pi_M$. Take z_0 of weight d_Z, j minimal such that π_j occurs as a pattern of z_0, at the check c say, and γ with $\gamma c_0 = c$. As $\rho_{\gamma c'} = \gamma \rho_{c'}$, the operator $z = \gamma^{-1} z_0$ has the same set of patterns and pattern π_j at c_0. □

For the cyclic codes the translations $i \mapsto i+1$ (in both blocks and on both kinds of checks) satisfy the hypotheses of Lemmas 2.4 and 2.5, with $w = 6$ and $M = 31$. For C600, `data/C600_automorphisms.json` contains three permutations of the qubits, computed as automorphisms of the Tanner graph with Sage; `src/codes.py` checks that each maps the X-check supports onto themselves and the Z-check supports onto themselves, and computes the orbits of the group they generate: four orbits of size 150, two in each block ($x^i y^j$ with j even, resp. odd).

3. The search

Here $\beta(T, X)$ is a lower bound for the size of every set E of free qubits meeting every check unsatisfied by T; both programs use the maximum of $\lceil u/h \rceil$, where u is the number of unsatisfied checks and h the largest number of unsatisfied checks through one free qubit, and the size of a greedily chosen family of unsatisfied checks with pairwise disjoint sets of free qubits (option `-b 2` of `cluster2`). Both programs take for $\mathcal{S}$ the Z-checks and the sums of two overlapping Z-checks (option `-m 2` of `cluster2`). Both programs apply the rules of Propagate only to the checks and the members of $\mathcal{S}$ that contain a qubit whose state has changed, and Grow does not put the last free qubit q_r into X; the proof of Proposition 3.1 uses only that every rule which is applied preserves compatibility, so it applies to the programs as well.

Proposition 3.1. *Let z be a nontrivial Z-logical operator with $|z| = d_Z \le W$, $T_0 \subseteq z$, $X_0 \cap z = \emptyset$ and $z \cap c \in \mathcal{P}_c$ for every check c, where every $\mathcal{P}_c$ consists of even sets and contains $\emptyset$. Then Algorithm 1 reports a set. Every reported set is a nontrivial Z-logical operator of weight at most W.*

Algorithm 1 Anchored search with exclusions

Require: H_X; a family $\mathcal{S} \subseteq \operatorname{rowsp} H_Z$; a bound W; disjoint sets $T_0 \neq \emptyset$, X_0; for every X-check c a family $\mathcal{P}_c$ of allowed intersections

$T \leftarrow T_0$, $X \leftarrow X_0$; **if** PROPAGATE **then** GROW
procedure PROPAGATE ▷ returns `false` on a contradiction
  repeat
    for every check c: if no $P \in \mathcal{P}_c$ satisfies $T \cap c \subseteq P \subseteq c \setminus X$, **return** `false`; otherwise put into T the qubits lying in all such P and into X the qubits of $c \setminus (T \cup X)$ lying in none
    for $s \in \mathcal{S}$ with $|T \cap s| > \lfloor |s|/2 \rfloor$: **return** `false`
    for $s \in \mathcal{S}$ with $|T \cap s| = \lfloor |s|/2 \rfloor$: put $s \setminus T$ into X
  until nothing changes
  return $|T| + \beta(T, X) \leq W$
procedure GROW
  if $\sigma(T) = 0$ **then** report T if it is nontrivial; **return**
  choose an unsatisfied check c; let $q_1, \ldots, q_r$ be its free qubits (not in $T \cup X$)
  for $i = 1, \ldots, r$ **do**
    if $q_i \in X$ **then continue**
    if $q_i \in T$ **then** GROW; **break**
    save the state; put q_i into T; **if** PROPAGATE **then** GROW; restore the state
    put q_i into X; **if not** PROPAGATE **then break**

Proof. A reported T has $\sigma(T) = 0$, is nontrivial and passed $|T| \leq W$. For the first assertion call a state (T, X) compatible if $T \subseteq z$ and $X \cap z = \emptyset$.

Propagation preserves compatibility and does not fail. For a check c, $P = z \cap c$ is allowed and satisfies $T \cap c \subseteq P \subseteq c \setminus X$, so the first rule finds such a P, and the qubits it puts into T (resp. X) lie in (resp. outside) $z \cap c$. The majority rules follow from Lemma 2.1(b): if $|T \cap s| = \lfloor |s|/2 \rfloor$, then $z \cap s = T \cap s$. The set $E = z \setminus T$ consists of free qubits and meets every unsatisfied check by Lemma 2.1(c), so $|T| + \beta(T, X) \leq |z| \leq W$; both bounds in β are lower bounds for $|E|$, since a free qubit lies in at most h unsatisfied checks and checks with disjoint sets of free qubits need distinct elements of E.

GROW called in a compatible state reports a set, by induction on $|z \setminus T|$. If $\sigma(T) = 0$, then $T = z$ by Lemma 2.1(a), and T is reported. Otherwise the chosen check c is unsatisfied, so some free qubit of c lies in z by Lemma 2.1(c); let q_j be the first. For $i < j$, $q_i \notin z$, so the state stays compatible when q_i is put into X, the loop is not left before $i = j$, and q_j is not put into X by a propagation. The state with $q_j \in T$ is compatible with a larger T, and the induction hypothesis applies. □

Corollary 3.2. *Assume the hypotheses of Lemma 2.4 (resp. Lemma 2.5). If Algorithm 1 reports nothing for every i with $T_0 = \{q_i\}$, $X_0 = X_i$ and $\mathcal{P}_c$ all even subsets of c (resp. for every j with $T_0 = \rho_0(\pi_j)$, $X_0 = c_0 \setminus T_0$ and $\mathcal{P}_c = \{\rho_c(\pi) : |\pi| \text{ even}, \ \pi \neq \pi_1, \ldots, \pi_{j-1}\}$), then $d_Z \geq W + 1$.*

Proof. Otherwise Lemma 2.4 (resp. Lemma 2.5) gives a minimum-weight z satisfying the hypotheses of Proposition 3.1 for one of the runs, which then reports a set. □

Relation to the literature. Growing a candidate from one position through unsatisfied checks is the irreducible-cluster technique of [DKP17, §V]. Algorithm 1 refines the complete search of Okada and Kasai [OK26, §V-A, Alg. 2, Lemma 2, Thm. 2], see also [LOMK26, App. B]

and [Lee26b, App. A]: explored siblings enter a forbidden set as in Grow, the two bounds combined in β prune, and a minimum-root convention restricts the anchors, which for a group of coordinate symmetries is Lemma 2.4 with $X_i = O_1 \cup \cdots \cup O_{i-1}$; this is also the anchor reduction of [CGP26, Suppl. Inf., §5.3], and [Lee26, App. C.4] uses one root per orbit of the automorphism group of the Tanner graph. The fieldnote [Cro26] on the qLDPC challenge board uses anchors at the two block origins, the exclusion of earlier candidates, a packing bound and the parity argument of Lemma 2.2, with two separately written programs. The canonical branching, the covering bound and the dominance bound are also in [CGP26, Suppl. Inf., eqs. (S39), (S41), (S42)], and the search without exclusions and with $\beta = \lceil u/3 \rceil$ is [RD26, Alg. 1, Thm. 4]. We have not found in these sources the propagation through the families $\mathcal{P}_c$, the exclusion of $s \setminus T$ in the majority rule, the use of sums of two checks in $\mathcal{S}$, or the pattern anchoring of Lemma 2.5.

4. Computations

The two programs share no code: implementation B implements [RD26, Alg. 1] with the exclusions, the propagation, the majority rule and the bound β of Algorithm 1, and implementation A was written separately, from a description of Algorithm 1 and of the proof of Proposition 3.1. Both follow the same argument, which Section 3 proves, so their agreement tests the implementations, not the argument. `src/vsearch_orb.c` (implementation A) supports the pattern anchoring of Lemma 2.5 (option `-K`, with c_0 the first check of the instance file and ρ_c the order in which the file lists the qubits of c; `src/codes.py` lists the X-check i of a cyclic code as $i + s_1, i + s_2, i + s_3, m + i + t_1, m + i + t_2, m + i + t_3$, where $\operatorname{supp} a = \{s_1, s_2, s_3\}$ and $\operatorname{supp} b = \{t_1, t_2, t_3\}$, so that $\rho_{\gamma c_0} = \gamma \circ \rho_0$ for every translation γ) and anchors with exclusions read from a file (option `-L`); `src/cluster2.c` (implementation B) runs one anchor at a time, either with nothing excluded or with the whole first block excluded (option `-R`), which is Lemma 2.4 with $X_i = \emptyset$ for the orbits in the first block and X_i the first block for those in the second. Both read the instance files written by `src/codes.py`. Both were validated on the gross code $[[144, 12, 12]]$ and the two-gross code $[[288, 12, 18]]$ of [B+24], whose distances are proved in [RD26] and certified in [CGP26]: nothing at $W = d - 2$, a logical operator of weight d at $W = d$ (`runs/validation.log`).

Theorem 4.1. *The codes C600, C504, B510, B558 and B434 have $d_X = d_Z =$ 26, 28, 26, 28 and 28, respectively.*

Proof. Upper bounds. The files `data/*_witnesses_Z.json` and `data/*_witnesses_X.json` contain nontrivial logical operators of the stated weights (83 and 70 for C600; one of each type for the board codes, taken from the board entries, where they were found by V. Russo); `src/codes.py` checks them. For C504 the Z-witness was found by implementation A at $W = 28$ (`runs/W_C504.log`, `runs/found_C504_W28.txt`), and the X-witness is its image under the permutation π of Lemma 2.3.

Lower bounds. All columns of H_X and H_Z have weight 3, so by Lemma 2.2 it suffices to exclude nontrivial logical operators of weight at most $d - 2$. For the cyclic codes $d_X = d_Z$ by Lemma 2.3, so it suffices to treat d_Z; for C600 both sides were searched. The runs in the table below report nothing at $W = d - 2$; by Corollary 3.2 the runs of each implementation together prove $d_Z \geq d - 1$, resp. $d_X \geq d - 1$, hence $\geq d$ by Lemma 2.2. Here Corollary 3.2 is applied as follows. Cyclic codes, implementation A: Lemma 2.5 for the translations. Cyclic codes, implementation B: Lemma 2.4 for the translations, whose orbits are the two blocks, with $q_1 = 0$, $X_1 = \emptyset$ and $q_2 = m$, X_2 the first block. C600: Lemma 2.4 for the group of Section 2, with $q_1, \ldots, q_4$ equal to $0, 1, 300, 301$ and O_i the orbit of q_i; $X_i = O_1 \cup \cdots \cup O_{i-1}$ for implementation

A (`instances/jobs_C600.txt`), and $X_1 = X_2 = \emptyset$, $X_3 = X_4$ the first block for implementation B. □

code	side, W	A: nodes, seconds	B: nodes, seconds	result
C600	Z, $W = 24$	$6.36 \cdot 10^7$, 345	$4.20 \cdot 10^8$, 412	nothing, $d_Z \geq 26$
C600	X, $W = 24$	$6.42 \cdot 10^7$, 392	$4.42 \cdot 10^8$, 501	nothing, $d_X \geq 26$
C504	Z, $W = 26$	$4.05 \cdot 10^8$, 1987	$5.25 \cdot 10^9$, 4873	nothing, $d_Z \geq 28$
B510	Z, $W = 24$	$9.16 \cdot 10^7$, 429	$1.08 \cdot 10^9$, 1115	nothing, $d_Z \geq 26$
B558	Z, $W = 26$	$3.29 \cdot 10^8$, 1594	$4.26 \cdot 10^9$, 4301	nothing, $d_Z \geq 28$
B434	Z, $W = 26$	$3.93 \cdot 10^8$, 1942	$5.21 \cdot 10^9$, 4911	nothing, $d_Z \geq 28$

A and B are the two implementations. Node counts are calls of Grow for A and generated states for B; for B, nodes and seconds are summed over the runs of `src/jobs.txt` for that line (`-a 0` and `-R` for the cyclic codes; `-a 0`, `-a 1`, `-a 300 -R` and `-a 301 -R` for C600); times are single-thread wall-clock seconds on an x86-64 workstation (gcc 16.2.1, `-O2`), with up to seven jobs running in parallel. The logs are in `runs/`.

Remark 4.2. The lower bounds are computer-assisted. Their correctness rests on Proposition 3.1, Corollary 3.2, and Lemmas 2.2 and 2.3 and on the correctness of the two programs; that two independently written programs agree on every run, and on the validation codes, makes an undetected error unlikely but does not exclude it. In the terms of the qLDPC challenge board the result is a solver verdict, not a checkable proof object.

5. Reproduction

`make` builds both programs and the instances and runs all searches (about 6.5 CPU-hours); `make validate` runs the validation; `make check` rechecks codes and witnesses. Requirements: a C compiler, `make` and Python 3 (standard library only). `SHA256SUMS` lists the checksums of sources, instances and logs.

Acknowledgment. The programs in `src/`, the computations and a first draft of this note were made with Claude Opus 5.5 (Anthropic) in Claude Code.

References

[B+24] S. Bravyi, A. W. Cross, J. M. Gambetta, D. Maslov, P. Rall, and T. J. Yoder, *High-threshold and low-overhead fault-tolerant quantum memory*, Nature **627** (2024), 778–782, arXiv:2308.07915.

[Board] Unitary Foundation, *qLDPC challenge*, `https://qecchallenge.com`, repository `https://github.com/unitaryfoundation/qldpc-challenge` (commit `8042f9da`, 9 November 2026).

[CGP26] B. Cheng, N. Guo, and F. Pan, *Certifying quantum code and circuit distances at scale*, arXiv:2611.08247, 2026.

[Cro26] C. Crocker, *360-12-24 is Bravyi et al.'s (30,6) BB code (explicit permutation), with uncertified exhaustive-search evidence for $d = 24$*, fieldnote of 13 October 2026 in [Board], file `fieldnotes/2026-10-13-bb360-12-exhaustive-d24.md`.

[DKP17] I. Dumer, A. A. Kovalev, and L. P. Pryadko, *Distance verification for classical and quantum LDPC codes*, IEEE Trans. Inform. Theory **63** (2017), 4675–4686, arXiv:1611.07164.

[GZ25] V. Guémard and G. Zémor, *Moderate-length lifted quantum Tanner codes*, arXiv:2502.20297, 2025.

[HL26] Y. Hirasaki and J. Y. Lee, *Lifting lifted product codes*, arXiv:2607.28621, 2026.

[Lee26] G. Lee, *Bicycle Race: small two-block codes on the Pareto frontier*, arXiv:2610.13880, 2026.

[Lee26b] J. Y. Lee, *Designing group-valued codes with full regular low-weight bases*, arXiv:2610.06820, 2026.

[LOMK26] J. Y. Lee, K. Okada, N. Maskara, K. Kasai, and H. Zhou, *Design principles for ultra-high-rate quantum codes*, arXiv:2609.30069, 2026.

[LP24] H.-K. Lin and L. P. Pryadko, *Quantum two-block group algebra codes*, Phys. Rev. A **109** (2024), 022407, arXiv:2306.16400.

[OK26] K. Okada and K. Kasai, *Pair-partition constructions for CPM-based quantum LDPC codes*, arXiv:2607.14091, 2026.

[RD26] M. Rowshan and S. Devitt, *Logical operator decomposition for distance analysis of bivariate bicycle codes*, arXiv:2609.39395, 2026.

[WLP23] R. Wang, H.-K. Lin, and L. P. Pryadko, *Abelian and non-abelian quantum two-block codes*, in: 2023 12th International Symposium on Topics in Coding (ISTC), IEEE, 2023, arXiv:2305.06890.